

Getters aur Setters

Do type ki properties

1. **Data property:** seedhi value rakhti hai (ab tak jo use kiye).
2. **Accessor property:** asal me function hoti hai jo value **padhne (get)** ya **set karne (set)** par chalti hai, lekin bahar se normal property jaisi dikhti hai.

Syntax

```
javascript

let user = {
  name: "John",
  surname: "Smith",

  get fullName() {
    return `${this.name} ${this.surname}`;
  },

  set fullName(value) {
    [this.name, this.surname] = value.split(" ");
  }
};

user.fullName; // "John Smith" (getter chala)
user.fullName = "Alice Cooper"; // setter chala
```

- Bahar se `user.fullName()` nahi, sirf `user.fullName` likhte hain.
- Sirf getter ho aur setter na ho, to value set karne par error aata hai. Yaani property read-only ho jaati hai.

Accessor descriptor

`defineProperty` me accessor ke liye `value` / `writable` ki jagah `get` aur `set` dete hain (`enumerable` aur `configurable` waise hi).

```
javascript

Object.defineProperty(user, "fullName", {
  get() { ... },
  set(value) { ... }
});
```

Ek property ya to data hogi ya accessor, dono nahi. `get` aur `value` saath doge to error.

Smart getters/setters

Setter me validation daal sakte ho. Asli value alag property (`_name`) me rakhte hain.

javascript

```
let user = {  
  get name() { return this._name; },  
  set name(value) {  
    if (value.length < 4) { console.log("Bahut chhota naam"); return; }  
    this._name = value;  
  }  
};
```

`_` se shuru hone wali property ko convention ke hisaab se bahar se nahi chhedte.

Compatibility me kaam aata hai

Maano pehle `age` store karte the, ab `birthday` store karte ho. Purana code `age` padhta hai, to `age` ko **getter** bana do jo birthday se nikaal de. Purana code bina badle chalta rahega.

Yaad rakho

- Getter/setter = function, par property jaise dikhte hain.
- Validation aur calculated ("virtual") properties ke liye best.